Université des sciences et de la technologie Houari-Boumédiène (USTHB)

SecureCorp Zero-Trust Access Control System

Realized by:

- KHOUAS Ihsene Noor
- BENYETTOU Hadil

Introduction
1. System Architecture
1.1 Global Architecture Overview
1.2 Authentication Flow (Kerberos-Inspired)
2. Codebase Structure and File Descriptions
3. Authorization Model: RBAC and ABAC
3.1 Role-Based Access Control (RBAC)
4. Dual-Version Methodology
5. Security Analysis and Attack Simulations
5.1 Attack 1 — Replay Attack
Vulnerability Description
Vulnerable Code
Exploit Sequence (attack_1_replay.py)
Secure Fix
5.2 Attack 2 — Ticket Tampering
Vulnerability Description
Vulnerable Code
Exploit Sequence (attack_2_tampering.py)
Secure Fix
Vulnerability Description
Vulnerable Code
Exploit Sequence (attack_3_privilege_escalation.py)
Secure Fix
5.4 Attack 4 — Unauthorized Cross-Department Access
Vulnerability Description
Vulnerable Code
Exploit Sequence (attack_4_cross_dept.py)
Secure Fix
5.5 Attack 5 — Brute-Force (No Rate Limiting / Account Lockout)
Vulnerability Description
Vulnerable Code
Exploit Sequence (attack_5_brute_force.py)
Secure Fix
6. Logging and Monitoring
7. API Reference
8. Tools, Technologies, and Implementation Choices
9.Docker Deployment
Conclusion

Introduction

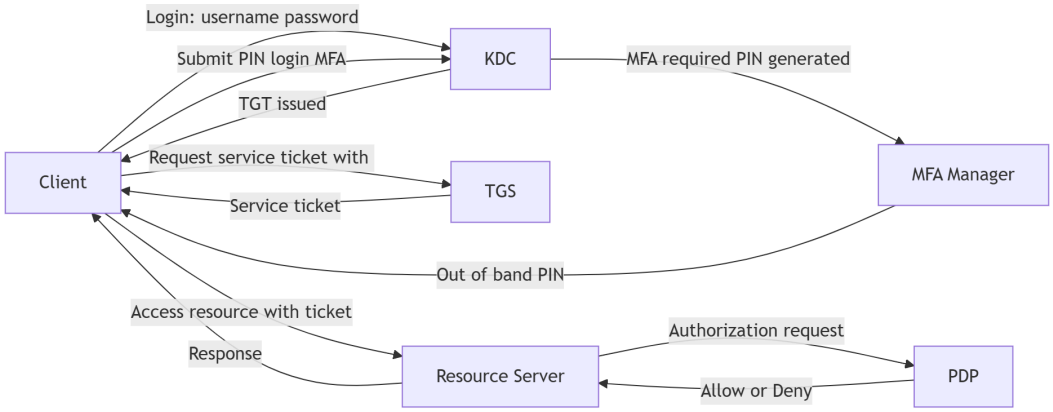
This report documents the design, implementation, and security analysis of a Zero-Trust Access Control System, developed as a fictional enterprise that manages sensitive internal data across four departments: HR, Finance, IT, and Operations. The system operates under a distributed microservices architecture with strict security enforcement, requiring a combination of modern authentication mechanisms, fine-grained authorization, and proactive attack mitigation.

The core objective was to design and implement a complete security stack from scratch, covering the following pillars: Kerberos-inspired ticket-based authentication, Role-Based Access Control (RBAC), Attribute-Based Access Control (ABAC), an external JSON policy engine, and Zero Trust enforcement ,the project demonstrates real-world attack vectors and their mitigations by producing two complete versions of the application: a deliberately vulnerable build and a hardened, secure build. This dual-version approach allowed us to run five attack demonstrations with clear before-and-after outcomes, which form the security analysis section of this report.

1. System Architecture

The system is built as a Python Flask application and follows a layered Zero Trust architecture. No component implicitly trusts any other. Every request must carry a cryptographically verified service ticket, and every access decision passes through the Policy Decision Point before any data is returned.

1.1 Global Architecture Overview



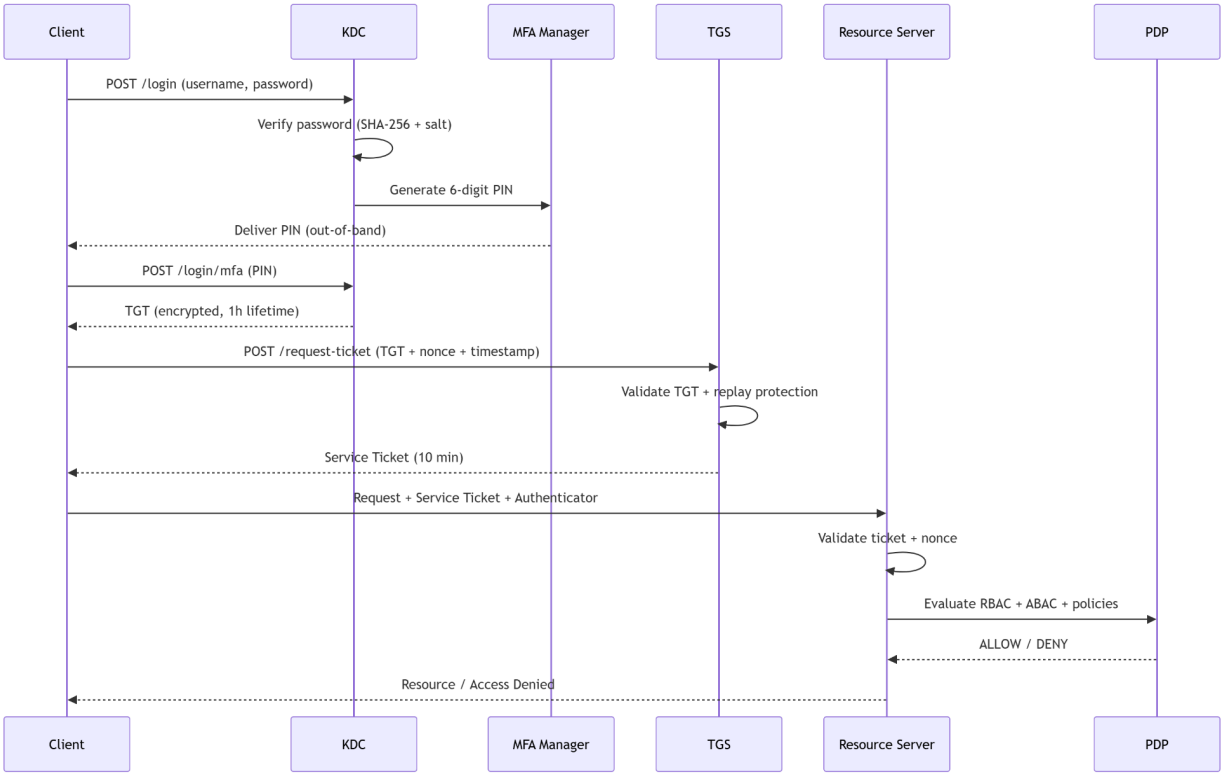
The architecture is divided into three major planes: Authentication (KDC + TGS), Authorization (PDP + Policy Engine), and Resource Access (Resource Server).

1.2 Authentication Flow (Kerberos-Inspired)

The authentication model is inspired by the MIT Kerberos protocol, adapted for a REST API context. In the secure build, it proceeds in four steps, with Multi-Factor Authentication (MFA) enforced between credential verification and TGT issuance:

- **Step 1 — Credential Verification:** The client submits credentials (username + password). The KDC verifies against users.json using SHA-256 hashed passwords with per-user salts. If valid, the server does not yet issue a TGT; instead, it generates a 6-digit MFA PIN via MFAManager, stores it with a short TTL, and returns {mfa_required: true} to signal that step 2 is required.

- **Step 2 — MFA Verification:** The PIN is delivered out-of-band via the mfa_server.py terminal (simulating an SMS gateway or authenticator app). The client submits the PIN to POST /login/mfa. On success, the KDC issues the Ticket-Granting Ticket (TGT), encrypted with the KDC master key. The TGT embeds the user's identity, role, department, clearance, location, a session key, and an expiry timestamp (1 hour).
- **Step 3 — Ticket Request:** The client presents the TGT alongside a one-time authenticator (nonce + timestamp) to the TGS. The TGS decrypts and validates the TGT, verifies the nonce is fresh (replay protection), and issues a Service Ticket encrypted with the TGS service key (10-minute lifetime).
- **Step 4 — Resource Access:** The client presents the Service Ticket in the X-Service-Ticket header. The Resource Server decrypts and validates the ticket, verifies the authenticator nonce, then delegates to the PDP for the final access decision.



1.3 Zero Trust Principles Applied

The system enforces Zero Trust across the following dimensions:

- **Never trust, always verify:** every resource request re-validates the ticket and re-queries the PDP, regardless of who the user is.
- **Least privilege:** each role grants only the minimum permissions (Employee: read only; Manager: read + write; Admin: read + write + delete). These are enforced in config.py and the PDP.
- **Assume breach:** all tickets are encrypted and HMAC-signed. Any modification is detected cryptographically before any attribute is read.
- **Explicit policy:** all authorization decisions are governed by external JSON policies (policies.json) that can be hot-reloaded without restarting the server.

- **Audit everything:** every authentication attempt, ticket issuance, access decision, and policy evaluation is written to a structured JSON log file (logs/security.log) and an in-memory ring buffer.
- **Multi-Factor Authentication:** a correct password alone is never sufficient to obtain a TGT. After credential verification, the KDC generates a cryptographically random 6-digit PIN (via MFAManager) and delivers it out-of-band through a separate channel (mfa_server.py terminal, simulating an SMS gateway or hardware token). The client must submit this PIN to POST /login/mfa before a TGT is issued. This ensures that a stolen password in isolation cannot grant access — an attacker would also need to compromise the second channel.

2. Codebase Structure and File Descriptions

The project is organized as a flat Python package. All modules reside in the same directory alongside the JSON data files. The table below maps each file to its responsibility in both the vulnerable and secure builds.

File	Category	Role & Key Behaviours
app.py	Core API	The central Flask application. Implements the KDC login endpoint (/login), TGS ticket exchange (/request-ticket), and three resource endpoints (GET, POST, DELETE /resource/:id). Houses the require_ticket() authorization decorator that decrypts the service ticket and invokes the PDP. In the vulnerable build, this decorator also reads role and clearance from the request body, enabling privilege escalation. Admin endpoints (/admin/*) and five /demo/* endpoints for live attack illustration are also defined here.
security.py	Cryptogra phy	Provides all cryptographic primitives. In the vulnerable build, encrypt_ticket() performs only base64 encoding of the JSON payload (the key is ignored), and decrypt_ticket() performs the inverse with no integrity check. In the secure build, both functions use the cryptography library's Fernet class (AES-128-CBC + HMAC-SHA256), making any bit-flip in the ticket raise an InvalidToken exception before any data is parsed. Also provides verify_password() (== vs secrets.compare_digest()), and the ReplayProtection class (stubbed in the vulnerable build; nonce dictionary with TTL-based purging in the secure build). Contains MFAManager for TOTP-style 6-digit PIN challenges in the secure build.

pdp.py	Authorization	The Policy Decision Point. Implements evaluate(), which first performs an RBAC check (role vs. operation) using role hierarchy from config.py, then chains through all ABAC policies loaded from policies.json. Each policy maps to a handler method (_pol_department_isolation, _pol_clearance, _pol_time_window, _pol_external_access, _pol_delete_restriction). In the vulnerable build, POL-001 (department isolation), POL-002 (clearance check), and POL-005 (delete restriction) are removed from the handlers dict; unrecognized IDs fall through to an unconditional ALLOW. In the secure build, all five handlers are present and unknown policy IDs default to DENY (fail-secure).
rate_limiter.py	Anti-Brute-Force	Implements account lockout and IP-based rate limiting. In the vulnerable build, the RateLimiter class is a complete stub: is_ip_blocked() and is_account_locked() always return (False, ''), and record_attempt() does nothing. In the secure build, the class tracks per-username failure counters (locking after 5 consecutive failures for 5 minutes) and per-IP timestamps using a sliding window (blocking after 20 failures in 60 seconds for 10 minutes). Thread-safe via threading.Lock.
config.py	Configuration	Single source of truth for all system-wide constants: KDC and TGS symmetric keys (32-byte byte strings), TGT lifetime (3600 s), service ticket lifetime (600 s), access window (08:00–18:00), ROLE_PERMISSIONS dict (Admin: read/write/delete; Manager: read/write; Employee: read), ROLE_HIERARCHY dict (Admin > Manager > Employee), CLEARANCE_RANK dict (public=0, confidential=1, secret=2), and server port (5000). Identical in both builds.
policies.json	ABAC Policies	External JSON file defining the five ABAC policies evaluated by the PDP. Each policy has an id, name, priority (evaluated in ascending order), effect (deny), description, and conditions. POL-001 enforces department isolation with an Admin exemption. POL-002 enforces clearance level vs. resource classification. POL-003 restricts access to business hours (08:00–18:00) with Admin exemption. POL-004 blocks external users from confidential or secret resources. POL-005 restricts delete operations to Admin only (Separation of Duties). The file can be hot-reloaded via POST /admin/reload-policies without restarting the server.

users.json	User Store	JSON user database. Each user record contains: role, department, clearance, location, a per-user salt, and a SHA-256 password hash (computed as SHA256(password:salt)). The file is loaded on every login request to avoid stale state. Contains 12 users spanning all role/department/clearance/location combinations for testing.
resources.json	Resource Store	JSON resource catalogue. Eight resources across HR, Finance, IT, and Operations departments, each with a name, department, classification (public/confidential/secret), and a data field (the sensitive payload). Used by all Resource Server endpoints.
logger_module.py	Audit Logging	SecurityLogger class implementing structured NDJSON logging. Writes one JSON object per line to logs/security.log via Python's logging module. Maintains an in-memory ring buffer (500 entries) accessible at /admin/logs. A separate attack event buffer collects WARNING and CRITICAL entries. Provides type-specific methods: log_auth_attempt, log_ticket_issued, log_ticket_invalid, log_access_decision, log_policy_evaluation, log_replay_attack, log_tamper_detected, log_privilege_escalation, log_suspicious. Identical in both builds.
external_policy_engine.py	External PDP	A standalone Flask microservice (port 5001) that exposes a REST interface to the same policy evaluation logic as pdp.py. Endpoints: POST /evaluate (returns decision + reason + trace), GET /policies (list loaded policies), POST /policies/reload (hot-reload), GET /health. Provides the same five policy handlers. Useful for scenarios where the PDP runs as a separate service (microservice architecture). Identical in both builds.
simulate.py	CLI Simulator	An interactive terminal application for exploring the access control model without running the HTTP server. Allows the operator to select a user, resource, and operation, and optionally override the current hour. Displays a step-by-step policy evaluation trace with ALLOW/DENY verdicts per policy. Also supports adding users, adding resources, and rendering a full access matrix (all users x all resources x all operations). Runs the full secure policy engine (all 5 policies).

admin_cli.py	Admin REPL	A command-line REPL for administrative operations against a running server. Supports: users list/add/delete/role/passwd/unlock, resources list/add/delete, policies list/reload (via HTTP), security status/reset-ip/logs (via HTTP). Reads/writes users.json and resources.json directly for offline operations; connects to the server for live operations (unlock, reload, logs).
web_ui.py	Web Dashboard	A browser-based management interface (separate Flask app or embedded). Provides graphical views of users, resources, policies, and access decisions for demonstration purposes.
mfa_server.py	MFA Terminal	A separate terminal process that polls GET /mfa/pending every 2 seconds and prints any pending 6-digit PIN challenges to the screen, simulating an out-of-band MFA channel (SMS gateway or authenticator device). Requires the secure build of app.py which implements the MFAManager.
attack_demo.py	Attack Suite	A single consolidated script that runs all five attack demonstrations sequentially against the vulnerable build. Each function (attack_1_replay through attack_5_brute_force) performs the exploit and prints color-coded EXPLOITED/BLOCKED results. Includes a summary table mapping each vulnerability to its affected file and root cause.

3. Authorization Model: RBAC and ABAC

3.1 Role-Based Access Control (RBAC)

Three roles in strict hierarchy: Employee (read only), Manager (read + write), Admin (read + write + delete). Permissions accumulate via inheritance, enforced by **_effective_permissions()** in pdp.py. POL-005 adds a Separation of Duties gate on delete, even for Admins.

3.2 Attribute-Based Access Control (ABAC) — Five Policies

Five policies evaluated in priority order after RBAC passes. First DENY terminates the chain immediately (fail-fast): POL-001 isolates users to their own department, POL-002 enforces clearance ranking, POL-003 restricts access to business hours (08:00–18:00), POL-004 blocks external users from non-public resources, POL-005 restricts delete to Admins only.

Role hierarchy is enforced via **_effective_permissions()** in pdp.py, which accumulates permissions from the role and all roles it inherits. This means an Admin implicitly has all Manager and Employee

permissions. Separation of Duties is enforced by POL-005: even if a user somehow acquires the Admin role, delete operations are also gated by the ABAC policy chain.

3.3 Policy Engine Design

Policies are stored externally in policies.json and loaded at startup via _load_policies(), sorted by priority. This separation of policy from code enables hot-reload via POST /admin/reload-policies without restarting the server. The PDP returns a structured policy trace alongside the final decision, enabling full audit transparency.

An additional external_policy_engine.py (port 5001) exposes the same evaluation logic over HTTP at POST /evaluate, supporting a microservice deployment model where the PDP runs as an independent service.

4. Dual-Version Methodology

The project implements two complete versions of the application: a deliberately vulnerable build and a hardened secure build. Rather than simply describing theoretical vulnerabilities, this approach allows each attack to be demonstrated concretely against the vulnerable build, and then blocked against the secure build. The same five attack scripts work identically against both builds; the difference lies entirely in the four files that were modified between versions.

Aspect	Vulnerable Build	Secure Build
security.py encrypt_ticket()	base64(JSON) only; key ignored	Fernet(AES-128-CBC + HMAC-SHA256)
security.py ReplayProtection	check_and_register() always True	Nonce dict + TTL window (300 s)
security.py verify_password()	== (timing-attack vulnerable)	secrets.compare_digest()
app.py require_ticket()	Reads role/clearance from body	Body values completely ignored
pdp.py handlers dict	POL-001, POL-002, POL-005 missing	All five handlers registered
pdp.py unknown policy IDs	Fall through to ALLOW	Default to DENY (fail-secure)

rate_limiter.py	All methods are no-ops (stub)	Full account lockout + IP blocking
Attack 1 Replay	EXPLOITED	BLOCKED
Attack 2 Tampering	EXPLOITED	BLOCKED
Attack 3 Priv. Escalation	EXPLOITED	BLOCKED
Attack 4 Cross-Dept	EXPLOITED	BLOCKED
Attack 5 Brute Force	EXPLOITED	BLOCKED

5. Security Analysis and Attack Simulations

5.1 Attack 1 — Replay Attack

Affected file: security.py — class ReplayProtection

Vulnerability Description

In the Kerberos protocol, a nonce (number used once) is included in the authenticator to prevent an intercepted ticket from being reused by an attacker. The ReplayProtection class is responsible for recording nonces that have been seen and rejecting any that appear more than once within a time window. In the vulnerable build, this mechanism is completely disabled: check_and_register() unconditionally returns True without storing the nonce, meaning any valid authenticator can be replayed indefinitely to obtain unlimited service tickets.

Vulnerable Code

```
class ReplayProtection:
      def check_and_register(self, nonce: str, timestamp: float) -> bool:
        # All checks removed — nonce is never stored
      return True   # always allows any nonce

# In app.py (request_ticket endpoint):
replay_protection.check_and_register(nonce, ts)   # return value IGNORED
```

Exploit Sequence (attack_1_replay.py)

- Step 1: The attacker (or a legitimate user whose session is intercepted) performs a valid TGS exchange. The nonce and timestamp of the authenticator are captured.

- Step 2: The attacker replays the identical nonce + timestamp in three subsequent POST /request-ticket requests, all using the same TGT.
- Step 3: In the vulnerable build, the server issues a new service ticket for each replay. The attacker now holds multiple valid service tickets obtained without ever re-authenticating. This is equivalent to session hijacking from a single intercepted network packet.

Secure Fix

```
class ReplayProtection:
      def check_and_register(self, nonce: str, timestamp: float) -> bool:
        now = time.time()
      with self._lock:
            self._purge(now)
            if abs(now - timestamp) > self._window: return False  # stale
            if nonce in self._used: return False                 # replay
            self._used[nonce] = now + self._window               # register
            return True

# In app.py: result is now checked and a 401 REPLAY_DETECTED returned
```

5.2 Attack 2 — Ticket Tampering

Affected file: security.py — encrypt_ticket() / decrypt_ticket()

Vulnerability Description

Service tickets carry all authorization attributes (role, department, clearance). For these attributes to be trustworthy, the ticket must be cryptographically protected against modification. In the vulnerable build, encrypt_ticket() simply base64-encodes the JSON payload. The encryption key passed as a parameter is completely ignored. Because base64 is a publicly known, reversible encoding, any attacker who obtains a service ticket can decode it, modify any field (e.g., upgrading role from Employee to Admin), re-encode it, and submit the forged token. The server's decrypt_ticket() performs only a base64 decode with no integrity check, so it accepts the modified token without raising any error.

Vulnerable Code

```
def encrypt_ticket(data: dict, key: bytes) -> str:
      raw = json.dumps(data).encode('utf-8')
      return base64.urlsafe_b64encode(raw).decode()  # key ignored!

def decrypt_ticket(token: str, key: bytes) -> dict:
      raw = base64.urlsafe_b64decode(token)         # no integrity check
      return json.loads(raw)
```

Exploit Sequence (attack_2_tampering.py)

- Step 1: carol (Employee / HR / clearance=public) logs in and obtains a service ticket. The ticket is a base64-encoded JSON object.
- Step 2: The attacker decodes the ticket (standard Python base64 decode), modifies the role field to Admin, clearance to secret, and department to IT, then re-encodes the modified JSON as base64.

- Step 3: The attacker sends a DELETE request to /resource/IT-001 (a secret IT resource) using the forged ticket. In the vulnerable build, the server accepts the token and executes the deletion as if carol were an Admin.

Secure Fix

```
from cryptography.fernet import Fernet, InvalidToken

def encrypt_ticket(data: dict, key: bytes) -> str:
	fernet = Fernet(_derive_fernet_key(key))
	return fernet.encrypt(json.dumps(data).encode()).decode()

def decrypt_ticket(token: str, key: bytes) -> dict:
	fernet = Fernet(_derive_fernet_key(key))
	raw = fernet.decrypt(token.encode())  # raises InvalidToken if tampered
	return json.loads(raw)
```

Fernet combines AES-128-CBC encryption with an HMAC-SHA256 message authentication code. Any single bit change to the ciphertext invalidates the HMAC. The decryption method raises an InvalidToken exception before returning any plaintext, which is caught by the caller and converted to a 401 TAMPER_DETECTED response.

5.3 Attack 3 — Privilege Escalation via Request Body Injection

Affected file: app.py — require_ticket() decorator

Vulnerability Description

Even with a legitimately-issued, cryptographically-valid service ticket, the require_ticket() decorator in the vulnerable build re-reads authorization attributes from the client-supplied request body and header. The following code overwrites the ticket's role and clearance with whatever the client sends:

Vulnerable Code

```
body = request.get_json(silent=True) or {}
if 'role' in body:
	ticket['role'] = body['role']           # VULNERABILITY
if 'clearance' in body:
	ticket['clearance'] = body['clearance']  # VULNERABILITY
if request.headers.get('X-Role'):
	ticket['role'] = request.headers['X-Role']  # VULNERABILITY
```

Exploit Sequence (attack_3_privilege_escalation.py)

- Step 1: carol logs in and obtains a legitimate Employee service ticket.
- Step 2: carol attempts to DELETE /resource/HR-001 with the real ticket. The PDP correctly returns 403 because Employee role lacks the delete permission.
- Step 3: carol re-sends the exact same DELETE request, this time including {"role": "Admin", "clearance": "secret"} in the JSON body. The require_ticket() decorator overwrites the ticket's role before calling the PDP. The PDP now evaluates the request as if carol were an Admin and returns ALLOW.

This attack is particularly dangerous because it requires no cryptographic knowledge and no network interception. Any authenticated user can escalate their privileges using a standard JSON body parameter.

Secure Fix

```
# All body/header overrides removed entirely.
# ticket is populated ONLY from the decrypted, HMAC-verified token.
# The body may still be read for resource payload data,
# but NEVER for authorization attributes.
```

5.4 Attack 4 — Unauthorized Cross-Department Access

Affected file: pdp.py — _evaluate_policy() handlers dict

Vulnerability Description

The PDP dispatches each policy ID to its handler via a Python dictionary. In the vulnerable build, POL-001 (department isolation) and POL-002 (clearance check) are simply absent from the handlers dictionary, and POL-005 (delete restriction) is also removed. When the PDP encounters a policy ID with no corresponding handler, it falls through to an unconditional ALLOW:

Vulnerable Code

```
handlers = {
      # 'POL-001': self._pol_department_isolation,  # REMOVED
      # 'POL-002': self._pol_clearance,             # REMOVED
      'POL-003': self._pol_time_window,
      'POL-004': self._pol_external_access,
      # 'POL-005': self._pol_delete_restriction,    # REMOVED
}
handler = handlers.get(pid)
if handler:
      return handler(...)
return {'decision': 'ALLOW', 'reason': f'Policy {pid} disabled'}  # VULN
```

Exploit Sequence (attack_4_cross_dept.py)

- Step 1: carol (Employee / HR / clearance=public) obtains a valid service ticket.
- Step 2: carol sends GET requests to FIN-002 (Finance/confidential), FIN-001 (Finance/secret), IT-001 (IT/secret), and OPS-001 (Operations/confidential).
- Step 3: In the vulnerable build, POL-001 is never evaluated, so the department check is never performed. POL-002 is also never evaluated, so the clearance check is also skipped. All four requests return HTTP 200 with the full resource data, leaking sensitive Finance, IT, and Operations data to an HR employee.

Secure Fix

```
handlers = {
      'POL-001': self._pol_department_isolation,   # RESTORED
      'POL-002': self._pol_clearance,               # RESTORED
      'POL-003': self._pol_time_window,
      'POL-004': self._pol_external_access,
      'POL-005': self._pol_delete_restriction,      # RESTORED
}
# Unknown policy IDs now default to DENY (fail-secure)
return {'decision': 'DENY', 'reason': 'Unknown policy — denied by default'}
```

5.5 Attack 5 — Brute-Force (No Rate Limiting / Account Lockout)

Affected file: rate_limiter.py — class RateLimiter

Vulnerability Description

The RateLimiter class is instantiated at module level and injected into the login endpoint. In the vulnerable build, the class is a complete no-op stub: every method returns immediately with a permissive default, and no state is ever maintained. There is no account lockout, no IP blocking, and no failure counter. The login endpoint accepts an unlimited number of incorrect password attempts without any delay or restriction.

Vulnerable Code

```
class RateLimiter:
      def is_ip_blocked(self, ip):     return False, ''
      def is_account_locked(self, user): return False, ''
      def record_attempt(self, ip, user, success): pass  # no-op
      def unlock_account(self, user):  pass
      def reset_ip(self, ip):          pass
```

Exploit Sequence (attack_5_brute_force.py)

- Step 1: The attacker targets the frank account and submits 10 commonly-used passwords from a wordlist in sequence. The server returns HTTP 401 for each wrong attempt but never issues a 429 lockout.
- Step 2: After all 10 failures, the attacker submits the correct password (Frank@2024). The server accepts it and returns a valid TGT.
- Step 3: In a real attack, the wordlist would be millions of entries long (rockyou.txt, etc.) and the attack could run fully automated. Without lockout, the attacker is guaranteed to eventually find the correct password.

Secure Fix

```
MAX_FAILURES  = 5
LOCKOUT_SECONDS = 300   # 5 min
IP_MAX_FAILURES = 20    # per 60-second sliding window
IP_BLOCK_SECONDS = 600  # 10 min

def record_attempt(self, ip, username, success):
      if success:
      self._accounts[username] = {'failures': 0, 'locked_until': 0}
      return
      rec = self._accounts.setdefault(username, {'failures': 0, 'locked_until':
0})
      rec['failures'] += 1
      if rec['failures'] >= self.MAX_FAILURES:
      rec['locked_until'] = time.time() + self.LOCKOUT_SECONDS
```

6. Logging and Monitoring

The SecurityLogger class (logger_module.py) provides comprehensive, structured audit logging throughout the system. Every security-relevant event is recorded as a JSON object (NDJSON format) to logs/security.log and simultaneously added to an in-memory ring buffer (500 entries maximum).

The /admin/logs endpoint (authenticated, Admin role required) returns the most recent N log entries from the in-memory buffer. The /admin/public-logs endpoint serves the same data without authentication for use by admin_cli.py. The /admin/attack-events endpoint returns only WARNING and CRITICAL severity events, making it easy to monitor for active attacks.

7. API Reference

Endpoint	Auth Required	Description
POST /login	None	KDC: verify credentials, issue TGT. Body: {username, password}. Returns: {tgt, session_key, user_info, tgt_expires_in}.
POST /mfa/verify	None (after /login)	Secure build only: submit 6-digit MFA PIN. Returns 200 + TGT on success.
POST /request-ticket	TGT required	TGS: exchange TGT + authenticator for a service ticket. Body: {tgt, service, authenticator{nonce, ts}}.
GET /resource/:id	Service Ticket	Read a resource. PDP evaluated. Returns resource data or 403.
POST /resource	Service Ticket (write)	Create a resource. PDP evaluated. Body: {department, classification, ...}.
DELETE /resource/:id	Service Ticket (delete)	Delete a resource. PDP evaluated. Admin role + POL-005 required.
GET /admin/logs	Service Ticket (Admin)	Return last N security log entries. Query param: ?n=50.

POST /admin/reload-policies	None (admin network)	Hot-reload policies.json into the PDP.
POST /admin/unlock/:username	None (admin network)	Clear account lockout for a username.
GET /admin/security-status	None (admin network)	Return current locked accounts and flagged IPs.
POST /demo/replay-attack	None	Demo endpoint: illustrates replay vulnerability/mitigation.
POST /demo/tamper-attack	None	Demo endpoint: illustrates tampering vulnerability/mitigation.
POST /demo/privilege-escalation	None	Demo endpoint: illustrates escalation vulnerability/mitigation.
POST /demo/unauthorized-access	None	Demo endpoint: illustrates cross-dept vulnerability/mitigation.
GET /health	None	Service health and build version.

8. Tools, Technologies, and Implementation Choices

Technology / Tool	Role in the Project
Python 3.11+	Primary implementation language for all server components.

Flask (Werkzeug)	Lightweight WSGI framework for the HTTP API. Chosen for simplicity and suitability for educational microservice demonstrations.
cryptography (Fernet)	AES-128-CBC encryption + HMAC-SHA256 authentication for service tickets. Fernet provides a self-contained, auditable cryptographic primitive with built-in key derivation and timestamp validation.
hashlib (SHA-256)	Password hashing. Passwords are stored as SHA-256(password:salt) with per-user random salts.
secrets module	Cryptographically secure random generation for session keys, nonces, MFA PINs, and salts. Replaces the insecure random module.
threading.Lock	Thread-safe nonce registry (ReplayProtection) and rate-limiter counters, since Flask may handle concurrent requests.
JSON (policies.json)	External policy storage enabling hot-reload and clean separation of policy from code. Policies are sorted by priority at load time.
Python logging	Structured NDJSON audit logging to file, with a concurrent in-memory ring buffer for API access.
requests library	HTTP client used by all attack scripts and the test suite.

9.Docker Deployment

The system was containerized using Docker to ensure portability and reproducibility across environments. A Docker Compose configuration was implemented to run the application as two separate services: the main Flask API and the MFA terminal, reflecting the system's distributed architecture. This setup allowed consistent deployment on both Windows and Linux, simplified dependency management, and enabled realistic simulation of inter-service communication within a Zero Trust environment. The application was configured to run on 0.0.0.0 to allow external access from the host machine, and images were successfully built and deployed locally, with optional publication to DockerHub for reuse.

Conclusion

This project successfully implements a complete Zero-Trust access control system, meeting all requirements set out in the assignment specification. The system demonstrates a working Kerberos-inspired authentication flow with encrypted, HMAC-signed tickets; a layered authorization model combining RBAC (role hierarchy) with ABAC (five context-sensitive policies); an external JSON policy engine with hot-reload capability; and comprehensive audit logging.

The dual-version methodology permitted the introduction of five specific vulnerabilities into the vulnerable build and then precisely fixing them in the secure build covering a broad and representative spectrum of real-world authentication and authorization weaknesses: replay attacks (nonce management), ticket tampering (cryptographic integrity), privilege escalation (client-controlled authorization attributes), broken access control (missing policy enforcement), and brute-force (lack of rate limiting). These correspond directly to several entries in the OWASP Top 10 (particularly A01: Broken Access Control, A02: Cryptographic Failures, and A07: Identification and Authentication Failures).

Future improvements would include: migrating from a flat JSON user store to a proper database with parameterized queries; adding mutual authentication (the client also verifying the server's identity); implementing token revocation lists for early TGT invalidation; and containerizing the application with Docker for reproducible deployment as outlined in the bonus requirements.